

libguestfs helps overcome this challenge. Since Anaconda installation is not done while virtual machines are deployed from pre-defined templates, using `kickstart %post` is not an option. libguestfs can be used to automatically make these required modifications inside a guest filesystem before the guest is deployed.

libguestfs is a library used to inspect, modify and manipulate virtual machine disk images. It supports almost all virtual disk formats like the raw image, vmdk (VMware image format), qcow2, LVM block devices, etc, and filesystems like ext2, ext3, ext4, btrfs, FAT, NTFS, etc.

The libguestfs tools

libguestfs provides some tools to manipulate the images efficiently. These are provided through the *libguestfs-tools* package in Fedora and can be used on the guest images non-interactively.

- virt-rescue:** Launches a shell in the guest image to do rescue operations like editing boot configuration files, modifying `initrd` images, etc.
- virt-edit:** Opens the specified file inside the guest image in your favourite editor.
- virt-inspector:** Gets basic details about the guest from its image, like the kernel version, modules loaded, mount points, details of applications installed in the guest, etc.

Other tools are *virt-cat*, *virt-ls*, *virt-rescue*, *virt-df*, *virt-tar* and *virt-win-reg*—I'm assuming readers will find the names self-explanatory. *virt-win-reg* is used to display Registry Entries from a Windows guest image.

The syntax of how to use *virt-cat* is as follows:

```
# virt-cat <guest image or block device> <file name>
```

For example:

```
# virt-cat f12.img /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.
localhost
::1         localhost localhost.localdomain localhost6 localhost6.
localhost6
```

Checking the man pages for the respective tools will provide more details on how to use them.

There are certain tools developed with the help of libguestfs. *virt-v2v* is one of them, and is used to migrate virtual machines running on one platform to another one (for example, virtual machines running on VMware to KVM). libguestfs is used extensively by this tool to modify the filesystem images to run appropriately on the target platform.

libguestfs can be used to inspect a guest filesystem while in offline mode and collect data for administrative purposes.

guestfish

guestfish uses libguestfs to launch a shell on the guest filesystem to help us modify the image efficiently, offline or online. It's recommended not to invoke it on running guests and make

modifications, due to the possibility of filesystem corruption.

You can invoke *guestfish* on online guests in read-only mode to view details. You can call it from a shell script, invoked from the command line directly, without launching an interactive shell to execute commands inside the guest image. You can also use it to launch an interactive shell on the libguestfs filesystem and execute commands from the shell. Examples of each method are available in its respective man page.

You need to use the `-i` switch to execute *guestfish* to launch an interactive shell:

```
# guestfish -i f12.img
```

```
Welcome to guestfish, the libguestfs filesystem interactive shell for
editing virtual machine filesystems.
```

```
Type: 'help' for help with commands
      'quit' to quit the shell
```

```
>>>
```

Typing `help` here will display all the available commands inside *guestfish* and running the `help <command>` will give more details about the specific command.

Troubleshooting

You can also use libguestfs to make modifications inside a guest image for troubleshooting purposes. This makes troubleshooting on a broken guest very easy. An administrator does not need to go into rescue mode to reinstall Grub, make modifications to boot loader configuration files, etc. Let us look at the simple example of a kernel upgrade rendering the system non-bootable. Reverting to the old well-known working kernel is as simple as running the following command:

```
# guestfish -i <guest image/ blockdevice>
```

...and editing the `/boot/grub/grub.conf` file to change the default kernel to boot.

In the same manner, we can reinstall the boot loader on the guest image if it somehow gets corrupted. There is no need to follow the old way of looking for a bootable CD and going to rescue mode to reinstall it. As a matter of fact, this way of recovering a broken production virtual machine becomes much faster compared to recovering a broken physical system. Let us have a look at how we can reinstall Grub on a Fedora 12 guest image.

Use *virt-df* on the image to view the filesystem details of the guest:

```
# virt-df f12.img
```

Filesystem	1K-blocks	Used	Available	Use%
f12.img:/dev/VolGroup1/vt_root	6821094	1346928	5128256	24.8%
f12.img:/dev/sda1	198337	21435	166602	26.8%